

Java Modifiers

[< Previous](#)[Next >](#)

Modifiers

By now, you are quite familiar with the `public` keyword that appears in almost all of our examples:

```
public class Main
```

The `public` keyword is an **access modifier**, meaning that it is used to set the access level for classes, attributes, methods and constructors.

We divide modifiers into two groups:

- **Access Modifiers** - controls the access level
- **Non-Access Modifiers** - do not control access level, but provides other functionality

Access Modifiers

For **classes**, you can use either `public` or *default*:

Modifier	Description	Try it
<code>public</code>	The class is accessible by any other class	Try it »
<i>default</i>	The class is only accessible by classes in the same package. This is used when you don't specify a modifier. You will learn more about packages in the Packages chapter	Try it »

For **attributes, methods and constructors**, you can use the one of the following:

Modifier	Description	Try it
<code>public</code>	The code is accessible for all classes	Try it »
<code>private</code>	The code is only accessible within the declared class	Try it »
<i>default</i>	The code is only accessible in the same package. This is used when you don't specify a modifier. You will learn more about packages in the Packages chapter	Try it »
<code>protected</code>	The code is accessible in the same package and subclasses . You will learn more about subclasses and superclasses in the Inheritance chapter	Try it »

Non-Access Modifiers

For **classes**, you can use either `final` or `abstract` :

Modifier	Description	Try it
<code>final</code>	The class cannot be inherited by other classes (You will learn more about inheritance in the Inheritance chapter)	Try it »
<code>abstract</code>	The class cannot be used to create objects (To access an abstract class, it must be inherited from another class. You will learn more about inheritance and abstraction in the Inheritance and Abstraction chapters)	Try it »

For **attributes and methods**, you can use the one of the following:

Modifier	Description
<code>final</code>	Attributes and methods cannot be overridden/modified
<code>static</code>	Attributes and methods belongs to the class, rather than an object
<code>abstract</code>	Can only be used in an abstract class, and can only be used on methods. The method does not have a body, for example <code>abstract void run();</code> . The body is provided by the subclass (inherited from). You will learn more about inheritance and abstraction in the Inheritance and Abstraction chapters
<code>transient</code>	Attributes and methods are skipped when serializing the object containing them
<code>synchronized</code>	Methods can only be accessed by one thread at a time
<code>volatile</code>	The value of an attribute is not cached thread-locally, and is always read from the "main memory"

Final

If you don't want the ability to override existing attribute values, declare attributes as `final` :

Example

[Get your own Java Server](#)

```
public class Main {  
    final int x = 10;  
    final double PI = 3.14;  
  
    public static void main(String[] args) {  
        Main myObj = new Main();  
        myObj.x = 50; // will generate an error: cannot assign a value to a final variable  
        myObj.PI = 25; // will generate an error: cannot assign a value to a final variable  
        System.out.println(myObj.x);  
    }  
}
```

Static

A `static` method means that it can be accessed without creating an object of the class, unlike `public`:

Example

An example to demonstrate the differences between `static` and `public` methods:

```
public class Main {
    // Static method
    static void myStaticMethod() {
        System.out.println("Static methods can be called without creating objects");
    }

    // Public method
    public void myPublicMethod() {
        System.out.println("Public methods must be called by creating objects");
    }

    // Main method
    public static void main(String[] args) {
        myStaticMethod(); // Call the static method
        // myPublicMethod(); This would output an error

        Main myObj = new Main(); // Create an object of Main
        myObj.myPublicMethod(); // Call the public method
    }
}
```

Abstract

An `abstract` method belongs to an `abstract` class, and it does not have a body. The body is provided by the subclass:

Example

```
// Code from filename: Main.java
// abstract class
abstract class Main {
    public String fname = "John";
    public int age = 24;
```

```

public abstract void study(); // abstract method
}

// Subclass (inherit from Main)
class Student extends Main {
    public int graduationYear = 2018;
    public void study() { // the body of the abstract method is provided here
        System.out.println("Studying all day long");
    }
}

// End code from filename: Main.java

// Code from filename: Second.java
class Second {
    public static void main(String[] args) {
        // create an object of the Student class (which inherits attributes and methods from Main)
        Student myObj = new Student();

        System.out.println("Name: " + myObj.fname);
        System.out.println("Age: " + myObj.age);
        System.out.println("Graduation Year: " + myObj.graduationYear);
        myObj.study(); // call abstract method
    }
}

```

[Try it Yourself »](#)